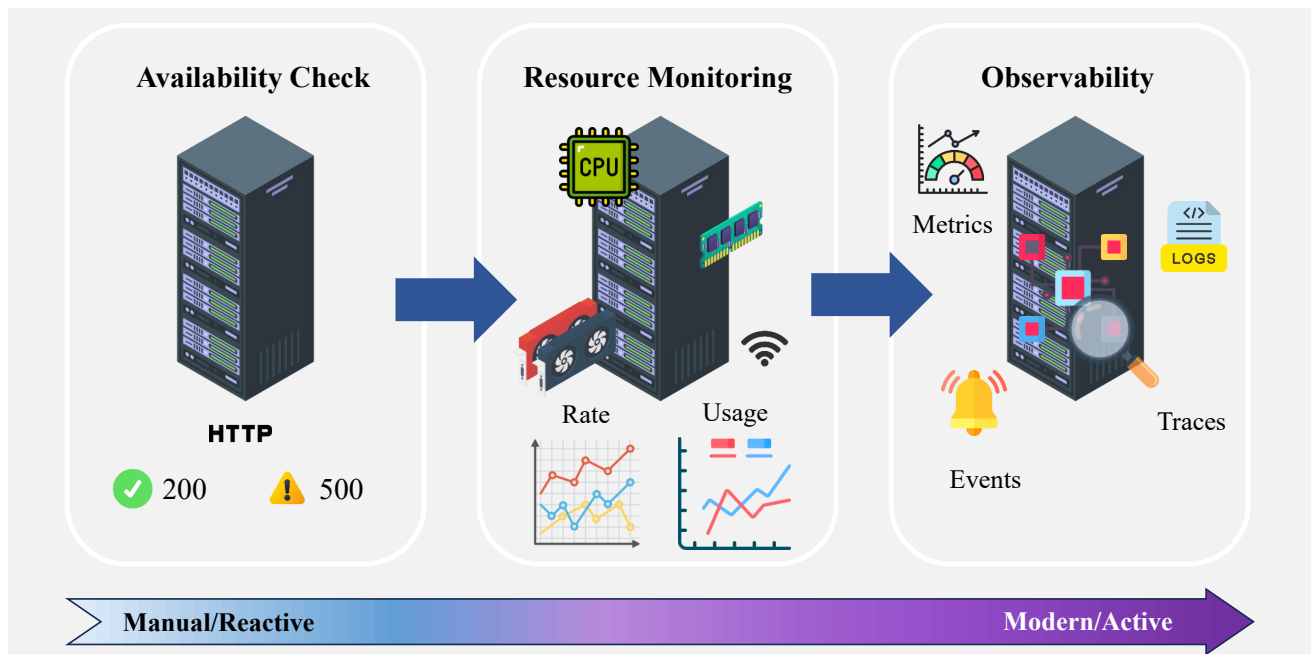


Tutorial: Giám sát hệ thống AI với Grafana và Prometheus

Dang-Nha Nguyen, Huu-Anh-Duong Ta, Quang-Vinh Dinh

I. Bối cảnh

Trong quá trình vận hành hệ thống, đặc biệt là các hệ thống Machine Learning (ML), nhiều doanh nghiệp bắt đầu với những phương pháp giám sát thủ công. Với cách tiếp cận này, nhìn thoáng qua có vẻ đơn giản và tiết kiệm chi phí ban đầu, nhưng thực tế khi hệ thống mở rộng về quy mô và độ phức tạp thì đây lại là nhược điểm khiến cho đội ngũ kỹ thuật khó khăn trong việc theo dõi, bảo trì hệ thống. Vậy đâu là những bất cập mà phương pháp thủ công không thể cải thiện và lý do tại sao chúng ta cần một tư duy giám sát hiện đại hơn? Chúng ta sẽ cùng đi qua 3 giai đoạn chính về nhu cầu giám sát hệ thống của kỹ sư vận hành qua từng thời kỳ:



Hình 1: Nhu cầu giám sát hệ thống qua từng giai đoạn.

- **Giai đoạn 1 - Server có hoạt động không:** Thường ở giai đoạn đầu, yêu cầu tối thiểu nhất của hệ thống đó là hoạt động được, có thể trả lời các requests, trả về đúng HTTP status code (ví dụ: 200 ok). Thế nhưng hạn chế rõ rệt là ngoài việc trả về log thông tin HTTP status, hầu như server không đưa ra thêm thông tin nâng cao nào, đôi khi status trả về là 200 nhưng người dùng cuối lại đối mặt với màn hình loading.
- **Giai đoạn 2 - Server phân bổ tài nguyên như thế nào:** Là sự nâng cấp của giai đoạn 1 khi nhận ra những nhược điểm đó, giai đoạn 2 các kỹ sư thường sẽ cài đặt hiển thị rõ các chỉ số tài nguyên hệ thống như CPU, RAM, Disk I/O, Network, ... Sự cải thiện này đã cung cấp đáng kể thông tin ở phía hệ thống, thế nhưng vẫn tiếp tục có nhiều vấn đề xảy ra mà thông tin ở server vẫn không đủ để xử lý như một số transaction khóa (deadlock) database, hay một số lỗi khác tới từ microservice, cho thấy ngoài sức khỏe phần cứng ra, server vẫn chưa nắm bắt được business health.
- **Giai đoạn 3 - Nhu cầu nắm bắt đầu đủ thông tin của hệ thống:** Không chỉ ngừng ở việc giám sát các chỉ số đã biết trước, các kỹ sư cần biết những thông tin đưa ra ý nghĩa "tại sao server không hoạt động" thay vì "server đang không hoạt động chỗ nào". Nhu cầu này được gọi với cái tên observability, với đặc điểm tập trung vào dữ liệu chi tiết: logs, metrics, traces và events.

Mục lục

I.	Bối cảnh	1
I.1.	Vấn đề thiếu thông tin đối với sự thay đổi	5
I.2.	Vấn đề "ClickOps và Configuration Drift	5
II.	Nền tảng lý thuyết	6
II.1.	Prometheus	6
II.1.1.	Kiến trúc và cơ chế pull	6
II.1.2.	Mô hình dữ liệu	7
II.1.3.	PromQL - Ngôn ngữ truy vấn dữ liệu	7
II.2.	Grafana	8
II.2.1.	Kết nối Data Source & Dashboards	8
II.2.2.	Variables & Templating	8
II.2.3.	Alerting	8
III.	Thực hành: Xây dựng hệ thống giám sát với Prometheus và Grafana	9
III.1.	Bước 1: Thiết lập môi trường với Docker	9
III.2.	Bước 2: Xây Dựng ML Model và Inference API	11
III.3.	Bước 3: Cấu hình Prometheus và khởi chạy	14
III.4.	Bước 4: Xây dựng Dashboard Grafana với Truy Vấn PromQL trên lưu lượng truy cập giả lập	14
IV.	Kết luận	19
V.	Tài liệu tham khảo	20

Bảng Giải Thích Thuật Ngữ

Thuật Ngữ	Giải thích
Business Health (server)	Mức độ ổn định, hiệu suất và khả năng phục vụ của hệ thống, phản ánh tình trạng hoạt động tổng thể của dịch vụ.
Microservice	Kiến trúc chia hệ thống thành các dịch vụ nhỏ, độc lập, có thể triển khai và mở rộng riêng biệt.
Deadlock	Trạng thái các tiến trình chặn nhau vĩnh viễn vì mỗi bên đều chờ tài nguyên mà bên kia giữ.
Mean Time To Recovery (MTTR)	Thời gian trung bình cần để khôi phục hệ thống sau khi xảy ra sự cố.
Mean Time To Detect (MTTD)	Thời gian trung bình từ lúc sự cố xảy ra đến khi được phát hiện.
Development	Môi trường dùng cho việc phát triển, thử nghiệm tính năng mới ở mức độ cá nhân hoặc nhóm nhỏ.
Staging	Môi trường mô phỏng production để kiểm thử toàn diện trước khi triển khai thực tế.
Production	Môi trường vận hành thật, phục vụ người dùng cuối và yêu cầu độ ổn định cao nhất.
Downtime	Khoảng thời gian hệ thống ngừng hoạt động và không thể phục vụ người dùng.
Data Drift	Hiện tượng phân phối dữ liệu đầu vào thay đổi theo thời gian, khiến mô hình giảm hiệu quả.
Provisioning	Quá trình chuẩn bị và cung cấp tài nguyên phần cứng hoặc phần mềm cho hệ thống hoặc dịch vụ.
Reproducible	Khả năng tái tạo chính xác môi trường, kết quả hoặc pipeline theo cùng một cấu hình và dữ liệu.
Orchestrate	Điều phối các tác vụ tự động theo trình tự logic để hoạt động hài hòa trong hệ thống.
Backfill	Chạy lại pipeline trên dữ liệu lịch sử để lấp đầy các khoảng thời gian chưa được xử lý.
Scrape	Quá trình thu thập số liệu từ các endpoint theo định kỳ để phục vụ giám sát.
Exporters	Các ứng dụng giúp xuất số liệu từ hệ thống hoặc dịch vụ dưới dạng Prometheus có thể thu thập.

Như vậy, để đáp ứng được nhu cầu observability, chúng ta cần nắm bắt chi tiết những nhược điểm:

I.1. Vấn đề thiếu thông tin đối với sự thay đổi

Trong các hệ thống phần mềm hoặc pipeline MLOps hiện đại, bản thân metrics vận hành (CPU, latency, error rate, F1 score, drift...) chỉ đóng vai trò phản ánh “điều gì đang xảy ra”, nhưng không trả lời được câu hỏi quan trọng hơn: tại sao nó lại xảy ra. Lý do cốt lõi nằm ở việc thiếu thông tin về các sự kiện thay đổi (Change Events) như cập nhật mã nguồn, deploy model mới, thay đổi tham số, thay đổi cấu hình môi trường, v.v.

Để hiểu rõ, chúng ta lấy một ví dụ đơn giản: trong khi Hệ thống đang vận hành một cách bình thường, đội phát triển đã deploy một bản vá lỗi. Ban đầu không hề có một lỗi nào, nhưng sau đó vào giờ cao điểm khi người dùng sử dụng nhiều, hệ thống tăng vọt error rate và cảnh báo. Lúc này đội ngũ vận hành sẽ phải chữa cháy rà soát lại network, database, load balancing, microservice, ... trong khi lỗi đến từ bản cập nhật vá lỗi. Do đó, các metrics thông số sẽ trở nên vô nghĩa khi không kèm theo ngữ cảnh (ai cập nhật, thông tin cập nhật, ...) sẽ không giúp ích được gì cho việc giảm thời gian Mean Time To Recovery (vấn đề này chỉ ảnh hưởng trực tiếp tới chỉ số MTTR mà không ảnh hưởng tới MTTD - Mean Time To Detect bởi các error đã được nắm bắt và cảnh báo bởi dashboard)

I.2. Vấn đề "ClickOps và Configuration Drift

Trước tiên, chúng ta cần hiểu về ClickOps là gì? ClickOps hiểu đơn giản là cách vận hành hệ thống bằng cách “click thủ công” trên UI (AWS Console, GCP UI, Kubernetes dashboard...), thay vì quản lý bằng code (Infrastructure-as-Code, GitOps). Nguy hiểm lớn nhất của ClickOps là không tạo ra ghi nhận lịch sử thay đổi. Người vận hành chỉ cần “click một lần” để đổi setting và toàn bộ hệ thống thay đổi nhưng không ai biết.

Với Configuration Drift, nó xảy ra khi cấu hình giữa các môi trường (development, staging, production) dần khác nhau theo thời gian vì có sự thay đổi thủ công, không đồng bộ, dẫn đến thông số ở dashboard vẫn tốt nhưng thực tế production thì không như vậy.

Ví dụ, chúng ta có một hệ thống phân loại với output probability được đặt threshold ở staging là 0.4 dùng để kiểm thử, nhưng ở production lại có giá trị là 0.5 do một ai đó thay đổi trực tiếp trong UI hoặc trong config của Server mà không có lịch sử commit git. Model được đánh giá tốt ở staging nhưng production vận hành lại gặp nhiều kết quả false negative, dẫn tới error rate tăng vọt. Trong khi đó sự thay đổi threshold không có tracking, audit log dẫn tới MTTR tăng mạnh, sự bất đồng bộ giữa production và staging cũng khiến cho nhiều vấn đề xung đột phát sinh.

Do đó, để giải quyết tận gốc hai vấn đề trên, chúng ta cần chuyển đổi từ tư duy giám sát thủ công sang Monitoring as Code. Thông qua bài viết này, chúng ta sẽ cùng tìm hiểu cách mà bộ đôi Prometheus và Grafana hiện thực hóa tư duy này. Mục tiêu không dừng lại ở việc hướng dẫn cài đặt, mà nhắm tới xử lý các vấn đề cốt lõi:

- Sử dụng kỹ thuật **Annotations** để giải quyết sự thiếu hụt thông tin, bằng cách tự động ghi lại các sự kiện như deploy lên biểu đồ, cung cấp ngữ cảnh ngay lập tức khi sự cố xảy ra.

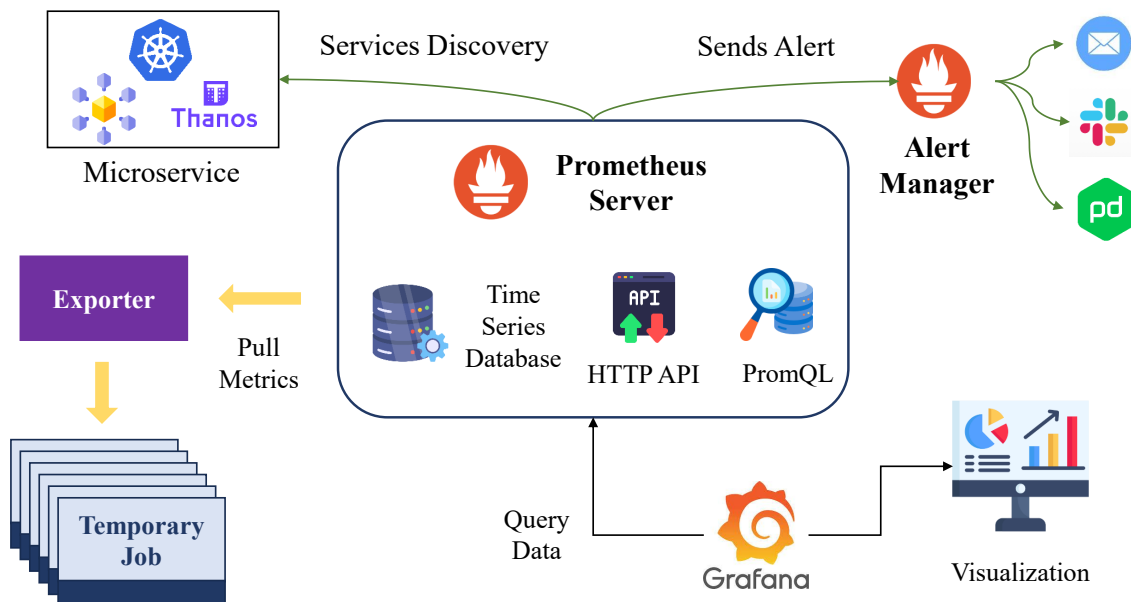
- Áp dụng kỹ thuật **Provisioning** (GitOps) để giải quyết vấn đề "ClickOps", bằng cách quản lý toàn bộ cấu hình dashboard, alert dưới dạng code (YAML/JSON) trong Git, đảm bảo tính nhất quán và khả năng khôi phục.

Với cộng đồng hỗ trợ cực kỳ lớn mạnh và đã trở thành chuẩn công nghiệp (de-facto standard) cho hệ thống giám sát hiện đại, Prometheus đảm nhiệm việc thu thập và lưu trữ metrics, trong khi Grafana là công cụ trực quan hóa mạnh mẽ. Khi được vận hành theo triết lý "Monitoring as Code", bộ đôi này sẽ giúp hệ thống giám sát của doanh nghiệp trở thành một phần mạnh mẽ, đáng tin cậy và có khả năng mở rộng trong hệ sinh thái MLOps.

II. Nền tảng lý thuyết

II.1. Prometheus

Prometheus được xem như trung tâm của hệ thống quan sát (observability stack) vì nó quyết định cách dữ liệu được thu thập, lưu trữ và truy vấn. Việc hiểu cơ chế hoạt động của Prometheus không chỉ giúp chúng ta thiết kế được hệ thống giám sát chính xác, mà còn nắm rõ lý do tại sao nhiều công cụ hiện đại (Grafana, Alertmanager, Thanos, Mimir...) lại dựa vào mô hình của Prometheus làm tiêu chuẩn chung. Phần này chúng ta sẽ tập trung làm rõ cấu trúc kỹ thuật cốt lõi của Prometheus, đặc biệt là cơ chế Pull, mô hình dữ liệu và PromQL – ba thành phần quyết định trải nghiệm sử dụng và khả năng mở rộng của toàn bộ hệ thống.



Hình 2: Cấu trúc tổng quát Prometheus.

II.1.1. Kiến trúc và cơ chế pull

Kiến trúc của Prometheus được thiết kế dựa trên sự chủ động và tính phi tập trung, với cốt lõi là cơ chế Pull Model. Khác với các hệ thống giám sát truyền thống thường thụ động chờ đợi dữ

liệu được gửi đến (Push), Prometheus chủ động kết nối và "kéo"(scrape) dữ liệu từ các target theo chu kỳ định trước. Cơ chế này mang lại lợi thế lớn về mặt kiểm soát luồng dữ liệu, giúp máy chủ Prometheus không bị quá tải bởi các đợt tấn công từ chối dịch vụ (DDoS) do lưu lượng metrics tăng đột biến, đồng thời dễ dàng phát hiện ra các node bị lỗi khi chúng ngừng phản hồi. Hơn nữa, Pull Model giúp việc debug trở nên dễ dàng hơn: chỉ cần truy cập endpoint /metrics để xem toàn bộ dữ liệu mà target đang expose mà không phải dò tìm trong pipeline đẩy dữ liệu như ở mô hình push.

Để hiện thực hóa việc thu thập này, hệ thống sử dụng các Exporters đóng vai trò như những cầu nối chuyển đổi dữ liệu thô thành định dạng Prometheus hiểu được. Ví dụ tiêu biểu là Node Exporter, chuyên thu thập thông tin mức hệ điều hành như CPU, RAM, network I/O; hoặc Blackbox Exporter, giúp kiểm tra HTTP, TCP, ICMP như một công cụ giám sát từ bên ngoài. Exporters giải quyết bài toán phổ quát: mỗi hệ thống có định dạng dữ liệu riêng, và Prometheus cần một ngôn ngữ chung để có thể thu thập dữ liệu nhất quán.

Trong môi trường động như Kubernetes, nơi các Pod thay đổi liên tục, việc cấu hình thủ công cho từng target gần như không khả thi. Đây là lý do Service Discovery trở thành thành phần không thể thiếu: Prometheus tự động phát hiện các service mới thông qua integration với Kubernetes API, Consul hoặc cloud provider. Nhờ đó, hệ thống luôn được cập nhật theo thời gian thực mà không cần chỉnh sửa cấu hình thủ công, giúp Prometheus thích ứng hoàn toàn với các môi trường có tính biến động cao.

II.1.2. Mô hình dữ liệu

Sức mạnh phân tích của Prometheus nằm ở Data Model đa chiều, nơi dữ liệu được lưu trữ dưới dạng chuỗi thời gian (Time Series) đi kèm với các định danh cụ thể. Một time series không chỉ bao gồm tên metric và giá trị số, mà còn được định nghĩa bởi tập hợp các cặp khóa-giá trị gọi là Labels (Nhãn); chính các labels này (ví dụ: `method="POST"`, `status="500"`) cho phép người quản trị thực hiện các truy vấn lọc sâu và phân tách dữ liệu theo nhiều chiều hướng khác nhau. Về mặt phân loại, Prometheus cung cấp bốn loại metric cơ bản phục vụ các mục đích đo lường riêng biệt. **Counter** là loại metric chỉ tăng (như tổng số request), dùng để đo lường tần suất tích lũy; **Gauge** là metric có thể tăng giảm tùy ý (như nhiệt độ CPU, dung lượng RAM), phản ánh trạng thái tại một thời điểm cụ thể. Trong khi đó, **Histogram** và **Summary** là những công cụ đặc lực để đo lường sự phân phối thống kê, chẳng hạn như độ trễ (latency) hay thời gian xử lý request, giúp nhận diện các vấn đề hiệu năng ở các phân vị (percentile) khác nhau. Như vậy, việc nắm vững mô hình dữ liệu và cách sử dụng labels chính xác là yếu tố tiên quyết để khai thác tối đa khả năng giám sát của Prometheus.

II.1.3. PromQL - Ngôn ngữ truy vấn dữ liệu

PromQL là ngôn ngữ truy vấn chức năng được thiết kế riêng để trích xuất và tính toán dữ liệu từ time-series database của Prometheus. Cú pháp của PromQL bắt đầu bằng việc sử dụng các Selectors để lọc ra tập hợp dữ liệu cần thiết dựa trên tên metric và các labels đi kèm, cho phép cô lập chính xác phạm vi vấn đề cần phân tích. Điểm đặc biệt của PromQL nằm ở hệ thống các hàm tính toán mạnh mẽ, điển hình là `rate()`, `irate()` và `increase()`. Trong đó, hàm `rate()`

đóng vai trò cực kỳ quan trọng khi làm việc với metric dạng **Counter**, bởi nó tính toán tốc độ thay đổi trung bình trên giây và tự động xử lý các trường hợp bộ đếm bị reset khi ứng dụng khởi động lại, giúp biểu đồ không bị gãy khúc sai lệch. Bên cạnh đó, các toán tử Aggregation như **sum by** hay **avg by** cho phép gộp nhóm dữ liệu từ hàng nghìn time series riêng lẻ thành một cái nhìn tổng quan theo các chiều mong muốn như theo server hoặc theo service. Kết luận lại, PromQL không chỉ là công cụ truy vấn mà là cầu nối logic biến dữ liệu thô thành các thông tin vận hành có ý nghĩa thực tiễn, đưa ra insight vận hành chính xác.

II.2. Grafana

Grafana giữ vai trò là lớp giao diện trực quan hóa dữ liệu từ Prometheus, biến những chuỗi thời gian phức tạp thành biểu đồ, bảng và dashboard trực quan. Mục tiêu chính của Grafana không phải thay đổi dữ liệu, mà là giúp con người hiểu dữ liệu dễ hơn bằng cách tổ chức, trình bày và hệ thống hóa thông tin một cách rõ ràng. Nhờ đó, Grafana trở thành công cụ tiêu chuẩn cho việc theo dõi hệ thống, phân tích sự cố và hỗ trợ ra quyết định.

II.2.1. Kết nối Data Source & Dashboards

Để thực hiện vai trò trực quan, điều kiện tiên quyết là sự kết nối giữa Data Source và Dashboards. Quá trình này bắt đầu bằng việc cấu hình Prometheus làm nguồn dữ liệu chính, cho phép Grafana gửi các câu lệnh PromQL và hiển thị kết quả trả về theo thời gian thực. Một Dashboard hiệu quả trong Grafana thường được thiết kế theo tư duy "từ tổng quan đến chi tiết" (Overview to Drill-down), bắt đầu bằng các chỉ số vĩ mô về sức khỏe hệ thống và cho phép người dùng đi sâu vào chi tiết khi phát hiện bất thường. Các thành phần hiển thị hay còn gọi là Panels rất đa dạng, từ Time series graph để theo dõi xu hướng, Stat (Single stat) để hiển thị con số tức thời, đến Gauge và Table để trình bày trạng thái tài nguyên. Chính sự linh hoạt trong việc kết hợp các loại panel và khả năng tương tác với data source đã biến Grafana trở thành công cụ không thể thiếu để "kể chuyện" về tình trạng hệ thống thông qua dữ liệu.

II.2.2. Variables & Templating

Tính năng Variables và Templating trong Grafana là giải pháp then chốt để giải quyết bài toán scalability trong việc quản lý dashboard. Thay vì phải tạo thủ công hàng trăm dashboard riêng lẻ cho từng server hay region, người quản trị có thể định nghĩa các biến (variables) để tạo ra các menu drop-down ngay trên giao diện, cho phép lọc dữ liệu linh hoạt theo tên máy chủ, vùng địa lý hoặc loại dịch vụ. Khi một giá trị được chọn từ menu, Grafana sẽ tự động thay thế biến đó vào trong tất cả các câu lệnh PromQL của dashboard, cập nhật toàn bộ dữ liệu hiển thị tương ứng. Cơ chế này tạo ra khái niệm Dynamic Dashboarding, nơi một dashboard mẫu duy nhất có thể tái sử dụng cho toàn bộ hạ tầng, giảm thiểu tối đa công sức bảo trì và nguy cơ sai sót cấu hình. Tóm lại, Variables và Templating là tính năng "must-have" giúp chuyển đổi Grafana từ một công cụ vẽ biểu đồ đơn thuần thành một nền tảng giám sát chuyên nghiệp và linh hoạt.

II.2.3. Alerting

Trong thực tế vận hành, không ai có thể dán mắt vào dashboard 24/7 để theo dõi hệ thống. Một hệ thống giám sát toàn diện không thể chỉ dựa vào việc con người quan sát biểu đồ mà phải

có khả năng tự động phát hiện và thông báo sự cố thông qua cơ chế Alerting. Quy trình này bắt đầu từ việc định nghĩa các **Alerting Rules** trong Prometheus dựa trên ngôn ngữ PromQL, ví dụ như thiết lập ngưỡng cảnh báo khi CPU vượt quá 80% trong vòng 5 phút liên tục. Một cảnh báo sẽ trải qua các trạng thái từ Inactive (bình thường), chuyển sang Pending (khi chạm ngưỡng nhưng chưa đủ thời gian quy định) và cuối cùng là Firing (khi sự cố đã được xác nhận). Sau khi cảnh báo được kích hoạt, nó được chuyển đến **Alertmanager** - thành phần chịu trách nhiệm xử lý hậu kỳ. **Alertmanager** không chỉ đơn thuần chuyển tiếp tin nhắn, mà nó thực hiện các tác vụ thông minh như grouping các cảnh báo liên quan, loại bỏ các cảnh báo trùng lặp (deduplication) để tránh spam, và routing thông báo đến đúng người nhận thông qua các kênh như Slack, Email hay PagerDuty. Nhờ sự phối hợp nhịp nhàng giữa **Rules** và **Alertmanager**, đội ngũ vận hành có thể phản ứng nhanh chóng với các sự cố thực sự nghiêm trọng mà không bị quá tải bởi các thông báo nhiễu.

III. Thực hành: Xây dựng hệ thống giám sát với Prometheus và Grafana

Trong phần thực hành này, chúng ta sẽ sử dụng một mô hình ML đơn giản từ scikit-learn để minh họa việc giám sát: mô hình phân loại hoa Iris trên dataset có sẵn. Mô hình sẽ được load từ một pretrained model cơ bản, đảm bảo inference nhanh chóng (dưới 100ms/request) để dễ dàng tạo nhiều request và quan sát metrics.

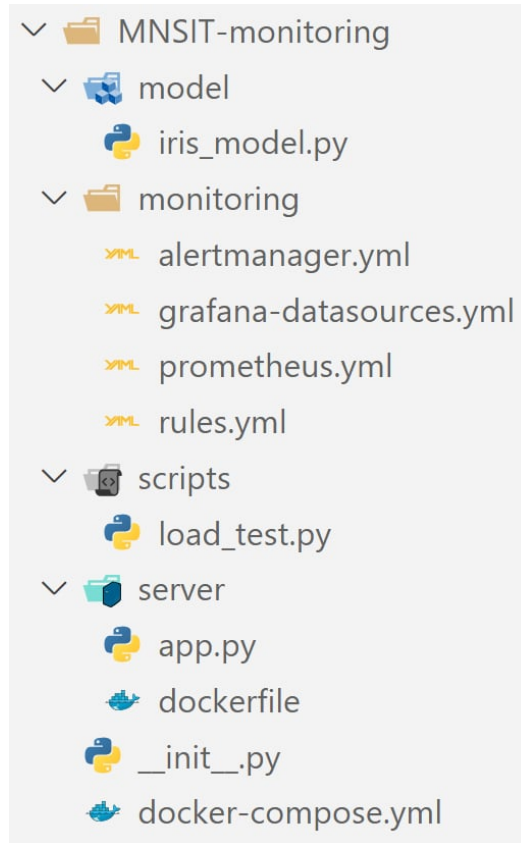
Mục tiêu của chúng ta là quan sát, theo dõi:

- **Request throughput:** Số lượng inference requests xử lý mỗi giây, giúp đánh giá tải hệ thống.
- **Latency:** Thời gian inference trung bình và phân vị (e.g., p95), để phát hiện chậm trễ do tải cao hoặc drift.
- **Error rate:** Tỷ lệ lỗi (ví dụ: invalid input hoặc model failure), tránh downtime.
- **Prediction distribution:** Phân bố các lớp dự đoán, giúp phát hiện data drift (e.g., input thay đổi so với training data).
- **Resource usage:** CPU/GPU memory, để tối ưu hóa tài nguyên trong production.

Objectives: Nắm vững cách instrument code ML với Prometheus client, expose metrics chuẩn OTel (OpenTelemetry), phân tích dữ liệu với PromQL, xây dựng Grafana dashboard từ scratch, thiết lập alerting rules cho latency/error spikes, và debug ML services dựa trên metrics – tất cả đều áp dụng linh hoạt trong pipeline MLOps.

III.1. Bước 1: Thiết lập môi trường với Docker

Triển khai cấu trúc dự án:



Hình 3: Cấu trúc dự án.

Ở bước này, chúng ta sẽ sử dụng Docker Compose để setup môi trường reproducible: Một container cho ML API (dựa trên Python/FastAPI), một cho Prometheus, và một cho Grafana. (Lý do: Tránh cài đặt thủ công phức tạp, dễ scale lên cloud, và phù hợp với MLOps practices như CI/CD)

docker-compose.yml

```

1 version: '3.11'
2 services:
3   ml-service:
4     build:
5       context: .
6       dockerfile: server/Dockerfile
7     ports:
8       - "8000:8000"
9     volumes:
10      - ./model:/app/model
11     networks:
12      - monitoring-net
13
14   prometheus:
15     image: prom/prometheus:latest
16     volumes:
17      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml
  
```

```

18     ports:
19         - "9090:9090"
20     networks:
21         - monitoring-net
22
23     grafana:
24         image: grafana/grafana:latest
25         ports:
26             - "3000:3000"
27         volumes:
28             - ../monitoring/grafana-datasources.yml:/etc/grafana/provisioning/datasources/all.
                yml
29         environment:
30             - GF_SECURITY_ADMIN_USER=admin
31             - GF_SECURITY_ADMIN_PASSWORD=admin
32         networks:
33             - monitoring-net
34
35     networks:
36         monitoring-net:

```

III.2. Bước 2: Xây Dựng ML Model và Inference API

Ở bước này, chúng ta sẽ chỉ tập trung vào inference phase vì monitoring thường áp dụng cho production serving trong MLOps.

Thực hiện download dataset, training model và lưu lại pretrained model chuẩn bị cho phần inference.

iris_model.py

```

1 from sklearn import datasets
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.model_selection import train_test_split
4 import joblib
5 import os
6
7 if not os.path.exists('./iris_model.pkl'):
8     print("Training model...")
9     iris = datasets.load_iris()
10    X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
        test_size=0.2)
11
12    model = LogisticRegression()
13    model.fit(X_train, y_train)
14    joblib.dump(model, './iris_model.pkl')
15    print("Model trained and saved.")
16
17 print("Loading model into memory...")
18 loaded_model = joblib.load('./iris_model.pkl')
19
20 def predict(features):

```

```

20     prediction = loaded_model.predict([features])[0]
21     return prediction.item()

```

Sử dụng FastAPI xây dựng API server cho mô hình với những metrics Prometheus cần thiết cho mục đích giám sát hệ thống đã đặt ra ở đầu bài:

- Counter: Theo dõi tổng requests/errors, dùng rate() cho throughput.
- Histogram: Phân bố latency, tính quantiles.
- Gauge: Status model hoặc resource (nếu cần biết thêm về GPU - thêm psutil).

app.py

```

1  import time
2  import logging
3  from fastapi import FastAPI, Request, HTTPException
4  from pydantic import BaseModel
5  from prometheus_client import Counter, Histogram, Gauge, make_asgi_app
6
7  try:
8      from model.iris_model import predict
9  except ImportError:
10     logging.warning("Could not import 'model.iris_model'. Using mock prediction for
11                                     testing.")
12
13     def predict(features):
14         return int(features[0]) % 3
15
16 logging.basicConfig(level=logging.INFO)
17 logger = logging.getLogger(__name__)
18
19 app = FastAPI()
20
21 REQUESTS_TOTAL = Counter(
22     'inference_requests_total',
23     'Total inference requests',
24     ['method', 'endpoint', 'status']
25 )
26
27 DURATION_SECONDS = Histogram(
28     'inference_duration_seconds',
29     'Inference duration',
30     ['method', 'endpoint'],
31     buckets=[0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0]
32 )
33
34 PREDICTION_OUTPUT = Counter(
35     'prediction_output_total',
36     'Prediction class distribution',
37     ['class'] # e.g., "0", "1", "2"
38 )
39
40 MODEL_LOADED = Gauge('model_loaded', 'Model load status')

```

```

39 MODEL_LOADED.set(1)
40
41 @app.middleware("http")
42 async def monitor_requests(request: Request, call_next):
43     method = request.method
44     path = request.url.path
45
46     if path == "/metrics":
47         return await call_next(request)
48
49     start_time = time.time()
50     status_code = 500
51
52     try:
53         response = await call_next(request)
54         status_code = response.status_code
55         return response
56     except Exception as e:
57         logger.error(f"Internal Server Error: {e}")
58         raise e
59     finally:
60         process_time = time.time() - start_time
61
62         REQUESTS_TOTAL.labels(
63             method=method,
64             endpoint=path,
65             status=status_code
66         ).inc()
67
68         DURATION_SECONDS.labels(
69             method=method,
70             endpoint=path
71         ).observe(process_time)
72
73 metrics_app = make_asgi_app()
74 app.mount("/metrics", metrics_app)
75
76 class FeaturesInput(BaseModel):
77     features: list[float] # sepal_length, sepal_width, petal_length, petal_width
78
79 @app.post("/predict")
80 def inference(input: FeaturesInput):
81     prediction = predict(input.features)
82     PREDICTION_OUTPUT.labels(str(int(prediction))).inc()
83     return {"prediction": int(prediction)}

```

Tạo docker file cho server:

dockerfile

```

1 FROM python:3.11
2 WORKDIR /app

```

```

3 COPY . /app
4 RUN pip install fastapi uvicorn scikit-learn==1.5.2 joblib prometheus_client
5 CMD ["uvicorn", "server.app:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]

```

III.3. Bước 3: Cấu hình Prometheus và khởi chạy

prometheus.yml

```

1 global:
2   scrape_interval: 15s # Pull mỗi 15s
3   evaluation_interval: 15s # Eval rules
4
5 scrape_configs:
6   - job_name: "ml-inference"
7     metrics_path: "/metrics"
8     static_configs:
9       - targets: ["ml-service:8000"]

```

Tiếp đó, chạy lệnh `docker-compose up -d` để start. Truy cập `localhost:8000/docs` cho API docs, `localhost:9090` cho Prometheus UI, và `localhost:3000` cho Grafana (login adminadmin).

III.4. Bước 4: Xây dựng Dashboard Grafana với Truy Vấn PromQL trên lưu lượng truy cập giả lập

Chúng ta sẽ mô phỏng quá trình truy cập bằng code để có thể tiến hành query metrics Prometheus:

dockerfile

```

1 import requests
2 import time
3 import random
4 import logging
5 from datetime import datetime
6
7 URL = "http://localhost:8000/predict"
8 LOG_FORMAT = "%(asctime)s - %(levelname)s - %(message)s"
9 logging.basicConfig(level=logging.INFO, format=LOG_FORMAT)
10 logger = logging.getLogger("Simulator")
11
12 def generate_normal_iris():
13     """Sinh dữ liệu Iris chuẩn, phân phối đều"""
14     return {
15         "features": [
16             round(random.uniform(4.3, 7.9), 1), # sepal_length
17             round(random.uniform(2.0, 4.4), 1), # sepal_width
18             round(random.uniform(1.0, 6.9), 1), # petal_length
19             round(random.uniform(0.1, 2.5), 1) # petal_width

```

```

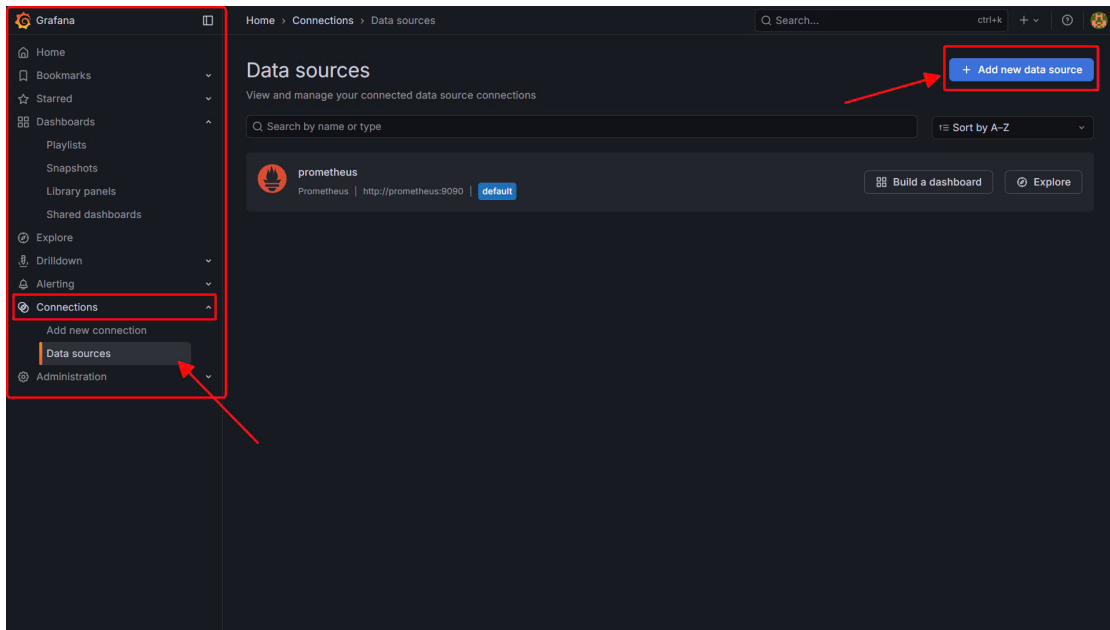
20     ]
21 }
22
23 def generate_drift_iris():
24     return {
25         "features": [
26             round(random.uniform(8.0, 10.0), 1), # Rất to
27             round(random.uniform(4.5, 6.0), 1),
28             round(random.uniform(7.0, 9.0), 1), # Rất dài
29             round(random.uniform(3.0, 5.0), 1)
30         ]
31     }
32
33 def generate_error_payload():
34     error_types = [
35         {"features": [1, 2]}, # Thiếu features
36         {"features": ["a", "b", "c", "d"]}, # Sai data type (str thay vì float)
37         {"wrong_key": [1, 2, 3, 4]}, # Sai key
38         {} # Empty
39     ]
40     return random.choice(error_types)
41
42 def run_scenario(name, duration_sec, delay_range, data_func):
43     logger.info(f"--- BẮT ĐẦU SCENARIO: {name} ({duration_sec}s) ---")
44     end_time = time.time() + duration_sec
45
46     while time.time() < end_time:
47         try:
48             payload = data_func()
49             response = requests.post(URL, json=payload, timeout=5)
50             if response.status_code == 200:
51                 # Chỉ in dấu chấm để đỡ spam màn hình khi chạy nhanh, hoặc in chi tiết nếu cần
52                 pred = response.json().get('prediction')
53                 print(f" {name} | Pred: {pred} | Input: {payload['features']}", end="\r\n")
54             else:
55                 print(f"\n {name} | Status: {response.status_code} | Error: {response.text}")
56
57         except Exception as e:
58             print(f"\n Connection Error: {e}")
59
60         time.sleep(random.uniform(*delay_range))
61     print("\n")
62
63 if __name__ == "__main__":
64     print(f"Starting ADVANCED traffic generation to {URL}...")
65     print("Cycle: Normal -> High Load -> Data Drift -> Errors -> Normal...")
66
67     try:
68         while True:
69             run_scenario(

```

```
70         name="NORMAL MODE",
71         duration_sec=30,
72         delay_range=(0.5, 1.5),
73         data_func=generate_normal_iris
74     )
75
76     run_scenario(
77         name="HIGH LOAD SPIKE",
78         duration_sec=15,
79         delay_range=(0.01, 0.05),
80         data_func=generate_normal_iris
81     )
82     run_scenario(
83         name="DATA DRIFT",
84         duration_sec=20,
85         delay_range=(0.3, 0.8),
86         data_func=generate_drift_iris
87     )
88
89     run_scenario(
90         name="ERROR STORM",
91         duration_sec=10,
92         delay_range=(0.2, 0.6),
93         data_func=generate_error_payload
94     )
95
96 except KeyboardInterrupt:
97     print("\n Stopped by user.")
```

Sau đó thực hiện chạy lệnh python scripts/load_test.py để thực hiện mô phỏng.
Các bước để tạo Dashboard Grafana:

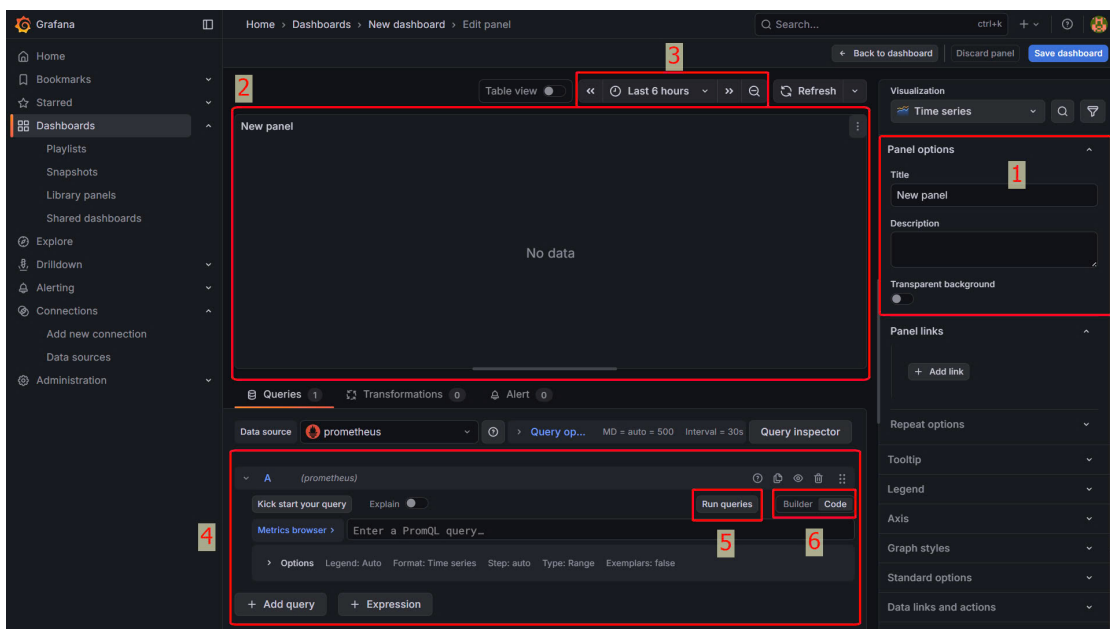
- **Bước 1:** Thực hiện kết nối Granafa với Data Source của Prometheus: Sau khi đăng nhập thành công Granafa và đổi mật khẩu, tiếp đó thực hiện mở thanh menu Granafa ở góc phía trên bên trái, chọn mục **Connections-> Data Sources -> Add new data source**



Hình 4: Tạo mới datasource

Tiếp đó, chọn loại databases là Prometheus. Thực hiện đặt tên, điền URL connection là <http://prometheus:9090> rồi chọn **Save & Test**

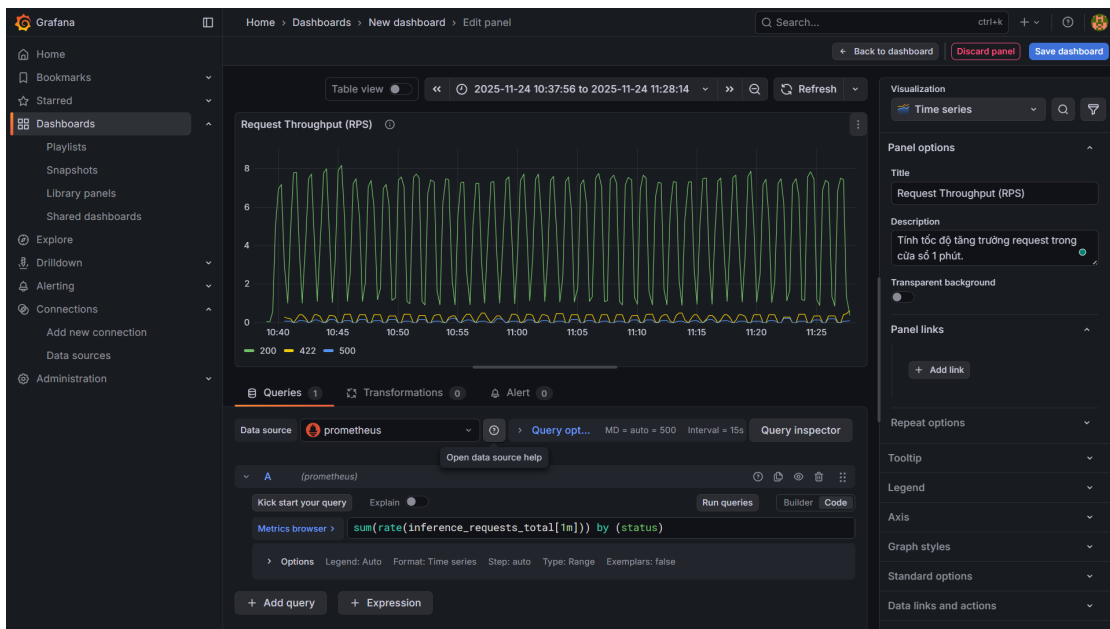
- **Bước 2:** Tạo dashboard với query PromQL: Cũng ở thanh menu của Granafa, chọn **Dashboard -> Create Dashboard -> Add Visualization** -> Chọn Data Source mà bạn vừa tạo, kết nối từ Prometheus. Từ đó chúng ta được giao diện như sau:



Hình 5: Giao diện tạo panel của Granafa

Trong đó:

- 1. Thông tin, mô tả nội dung panel
 - 2. Trực quan hóa dữ liệu metric bằng biểu đồ
 - 3. Range thời gian của dữ liệu
 - 4. Tạo query metric: Chúng ta có thể tạo bằng giao diện hoặc PromQL
 - 5. Nút thực hiện query
 - 6. Hai lựa chọn: query bằng UI hoặc bằng PromQL
- **Bước 3:** Thực hiện tạo 1 panel bằng lệnh PromQL: `sum(rate(inference_requests_total[1m])) by (status)` (Tính tốc độ tăng trưởng request trong cửa sổ 1 phút). Lưu ý, để có thể dùng lệnh trên, chúng ta phải đổi mode query của panel từ **Builder** sang **Code**



Hình 6: Ví dụ về khởi tạo Pabel: Request Throughput (RPS)

Tương tự, chúng ta trở về giao diện dashboard, tiếp tục thực hiện với các lệnh bên dưới cho từng panel mới được tạo:

- **Latency (P95 & Average):** `histogram_quantile(0.95, sum(rate(inference_duration_seconds_bucket[5m])) by (le)), sum(rate(inference_duration_seconds_sum[5m])) / sum(rate(inference_duration_seconds_count[5m]))`
- **Error Rate (%)**: `sum(rate(inference_requests_total{status~"5.."}[5m])) / sum(rate(inference_requests_total[5m])) * 100`
- **Prediction Distribution (Detect Drift)**: `sum(rate(prediction_output_total[1h])) by (class)`
- **CPU Usage**: `rate(process_cpu_seconds_total[1m])`
- **Memory Usage (RAM)**: `process_resident_memory_bytes`



Hình 7: Kết quả cuối cùng sau khi thực hiện

Ngoài ra, chúng ta còn có những biểu đồ tự động tạo theo metric ở phần **Drilldown** -> **Metrics**.

IV. Kết luận

Qua thực hành, chúng ta đã đạt được mục tiêu ban đầu đề ra: xây dựng một hệ thống giám sát end-to-end cho mô hình machine learning trong môi trường production, với khả năng quan sát toàn diện các khía cạnh cốt lõi của một inference service – từ request throughput, latency, error rate, prediction distribution cho đến resource usage. Qua việc tự tay instrument code bằng Prometheus client, cấu hình scrape, viết PromQL, dựng Grafana dashboard, chúng ta đã phần nào nắm vững cách thu thập và trực quan hóa metrics.

Hệ thống giám sát không còn là một công cụ đơn giản mà trở thành một phần không thể tách rời của pipeline MLOps: có thể tái tạo hoàn toàn, có lịch sử thay đổi rõ ràng, và luôn cung cấp đủ context để bất kỳ thành viên nào trong nhóm cũng có thể debug và phản ứng nhanh chóng. Đó chính là sự khác biệt giữa một hệ thống monitoring hoạt động và một hệ thống observability thực sự – nền tảng để đưa machine learning từ thử nghiệm sang vận hành tin cậy ở quy mô lớn.

V. Tài liệu tham khảo

- [1] K. Le. “[monitor] cách giám sát hệ thống đơn giản với prometheus và grafana”. Accessed: 2025-11-24. [Online]. Available: <https://viblo.asia/p/monitor-cach-giam-sat-he-thong-don-gian-voi-prometheus-va-grafana-BQyJKjW54Me>.
- [2] D. Murali. “Introduction to monitoring with prometheus & grafana”. Accessed: 2025-11-24. [Online]. Available: <https://medium.com/@dineshmurali/introduction-to-monitoring-with-prometheus-grafana-ea338d93b2d9>.
- [3] Unknown. “Introduction to monitoring with prometheus grafana”. Accessed: 2025-11-24. [Online]. Available: <https://www.youtube.com/watch?v=moLWjeXoVso>.
- [4] Grafana Labs. “Get started with grafana and prometheus”. Accessed: 2025-11-24. [Online]. Available: <https://grafana.com/docs/grafana/latest/getting-started/getting-started-prometheus/>.